Project2 算法分析说明书

一、简介

本代码实现了一个基于栅格地图的 A* 路径搜索算法，支持多种启发式函数（曼哈顿距离、欧 x 几里得距离、切比雪夫距离）和移动方向策略（四方向、八方向）。用户只需提供地图数据、起点与终点坐标，即可获得从起点到终点的最短路径点序列。代码结构清晰，易于集成到自动驾驶、机器人导航等项目中。

二、环境要求

Python 3.6 及以上版本

标准库：heapq、math、typing（无需额外安装）

可选库：matplotlib、numpy（用于可视化，不安装不影响算法运行）

三、代码结构

Node 类表示搜索树中的一个节点。

属性：x, y：节点在地图中的坐标。

g：从起点到当前节点的实际代价。

h：启发式估计代价（当前节点到目标点的估计距离）。

parent：父节点指针，用于回溯路径。

方法：f()：返回总代价 $f=g+h$。

__lt__：使节点可在优先队列中按 f 值排序。

__eq__, __hash__：使节点可被集合（set）使用。

AStar 类表示封装 A*算法的主体逻辑。

初始化方法：__init__(self, grid, start, goal, heuristic='manhattan', movement='4dir')：用于接收地图、起点、终点、启发式类型、移动模式。

主要方法：heuristic(node)：计算节点的启发式值 h。

is_valid(x, y)：检查坐标是否在地图内且可通行（值为 0）。

search()：执行 A* 搜索，返回路径列表（List[Tuple[int, int]]），若无解返回 None。

reconstruct_path(node)：从终点节点回溯，生成路径坐标列表。

四、使用步骤

1.进行参数定义

为了实现 A*路径规划算法的数据结构与逻辑，首先需定义 Node 类和 AStar 类以及它们的方法（即第四部分完整案例代码开始到#案例输入部分）

2.准备地图数据

地图用一个二维列表表示，每个元素代表一个栅格：

0：可通行区域

1：障碍物（不可通行）

例如一个 10×10 的地图，中间有一堵垂直的墙：

```
grid = [
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]
```

3.设置起点和终点

坐标格式为(x, y)，其中 x 表示行索引（从上到下），y 表示列索引（从左到右）。例如起点(0,0)左上角，终点(9,9)右下角。

```
start=(0,0)
goal=(9,9)
```

4.创建 AStar 对象

```
astar=AStar(grid, start, goal, heuristic='manhattan', movement='4dir')
```

参数说明：

heuristic：启发式函数，可选 'manhattan'、'euclidean'、'chebyshev'。

movement：移动方向，可选'4dir'（四方向）或'8dir'（八方向）。

5.执行搜索

```
path=astar.search()
```

如果找到路径，path 是一个列表，包含从起点到终点的所有坐标（包括起点和终点）。如果无解，返回 None。

6.输出路径结果

```
if path:
    print("路径长度:", len(path))
    print("路径点:", path)
else:
    print("未找到路径")
```

五、完整示例

```
import heapq
import math
from typing import List, Tuple, Optional
class Node:
    def __init__(self, x: int, y: int, g: float = 0, h: float = 0, parent: Optional['Node']
= None):
        self.x = x
        self.y = y
        self.g = g
        self.h = h
        self.parent = parent
    def f(self) -> float:
        return self.g + self.h
    def __lt__(self, other):
        return self.f() < other.f()
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __hash__(self):
        return hash((self.x, self.y))
class AStar:
    def __init__(self, grid: List[List[int]], start: Tuple[int, int], goal: Tuple[int, int],
                 heuristic: str = 'manhattan', movement: str = '4dir'):
        self.grid = grid
        self.start = Node(start[0], start[1])
        self.goal = Node(goal[0], goal[1])
        self.heuristic_type = heuristic
        self.movement = movement

        if movement == '4dir':
            self.neighbors = [(1,0), (-1,0), (0,1), (0,-1)]
        elif movement == '8dir':
            self.neighbors = [(1,0), (-1,0), (0,1), (0,-1),
                              (1,1), (1,-1), (-1,1), (-1,-1)]
        else:
```

```
            raise ValueError

    def heuristic(self, node: Node) -> float:
        dx = abs(node.x - self.goal.x)
        dy = abs(node.y - self.goal.y)

        if self.heuristic_type == 'manhattan':
            return dx + dy
        elif self.heuristic_type == 'euclidean':
            return math.hypot(dx, dy)
        elif self.heuristic_type == 'chebyshev':
            return max(dx, dy)
        else:
            raise ValueError

    def is_valid(self, x: int, y: int) -> bool:
        if x < 0 or x >= len(self.grid) or y < 0 or y >= len(self.grid[0]):
            return False
        return self.grid[x][y] == 0

    def search(self) -> Optional[List[Tuple[int, int]]]:
        self.start.h = self.heuristic(self.start)
        open_set = []
        closed_set = set()
        heapq.heappush(open_set, self.start)
        best_g = { (self.start.x, self.start.y): 0 }
        while open_set:
            current = heapq.heappop(open_set)
            if current == self.goal:
                return self.reconstruct_path(current)
            closed_set.add(current)
            for dx, dy in self.neighbors:
                nx, ny = current.x + dx, current.y + dy
                if not self.is_valid(nx, ny):
```

```
                continue
            neighbor = Node(nx, ny)
            if neighbor in closed_set:
                continue
            step_cost = math.hypot(dx, dy) if self.movement == '8dir' else 1.0
            tentative_g = current.g + step_cost
            if (nx, ny) not in best_g or tentative_g < best_g[(nx, ny)]:
                best_g[(nx, ny)] = tentative_g
                neighbor.g = tentative_g
                neighbor.h = self.heuristic(neighbor)
                neighbor.parent = current
                heapq.heappush(open_set, neighbor)
        return None
    def reconstruct_path(self, node: Node) -> List[Tuple[int, int]]:
        path = []
        while node:
            path.append((node.x, node.y))
            node = node.parent
        return path[::-1]
#案例输入
if __name__ == "__main__":
    grid = [
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    ]
    start = (0, 0)
```

```
goal = (9, 9)
print("曼哈顿启发式")
astar = AStar(grid, start, goal, heuristic='manhattan', movement='4dir')
path = astar.search()
if path:
    print("路径长度:", len(path))
    print("路径点:", path)
else:
    print("未找到路径")

print("欧几里得启发式")
astar = AStar(grid, start, goal, heuristic='euclidean', movement='8dir')
path = astar.search()
if path:
    print("路径长度:", len(path))
    print("路径点:", path)
else:
    print("未找到路径")
```

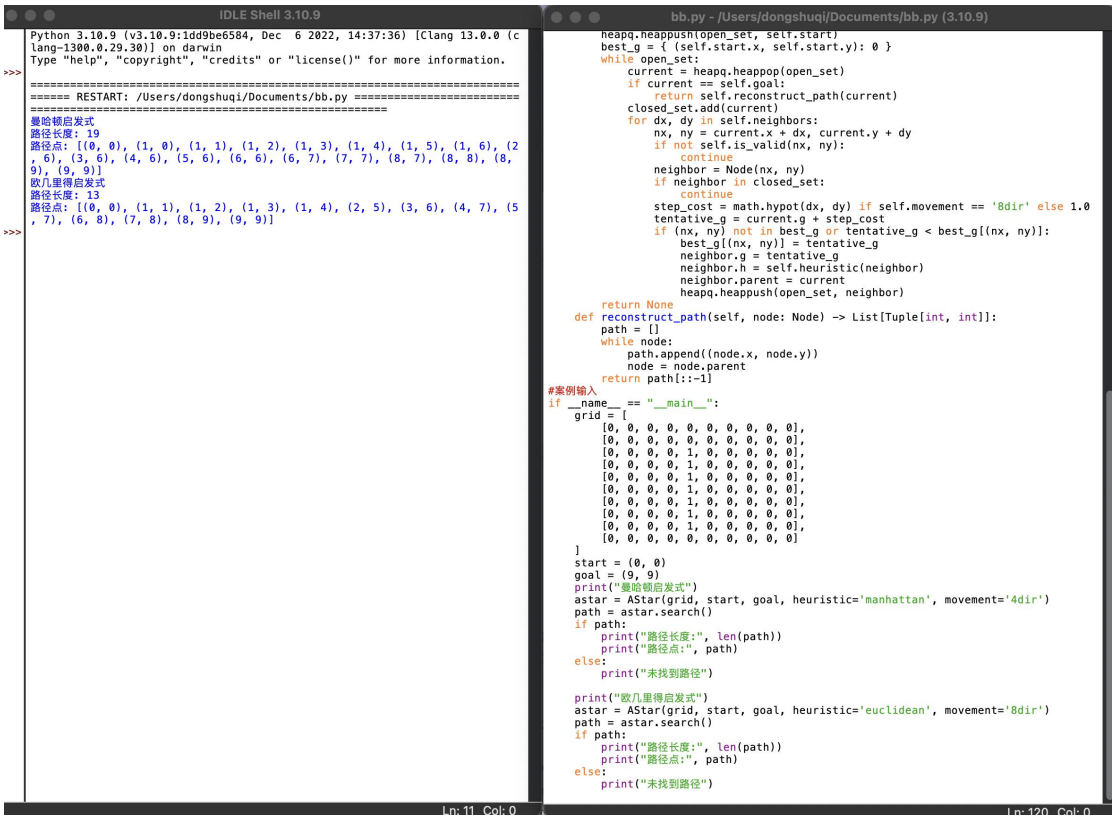

图 1 python 运行结果图

六、启发式函数实现

实现了三种常用的启发式函数：曼哈顿距离：$h = |dx| + |dy|$，适用于四方向移动，可采纳且一致，能保证最优性。欧几里得距离：$h = \sqrt{(dx^2 + dy^2)}$，适用于八方向移动，计算精确但可能高估，需注意可采纳性。切比雪夫距离：$h = \max(|dx|, |dy|)$，适用于八方向移动，允许对角线移动时更符合实际代价。可通过参数 heuristic 选择，满足不同场景需求。

七、算法核心步骤

在 A*算法的执行过程中，首先将起点节点加入优先队列 open_set，并记录其最佳 g 值；随后进入循环，每次从 open_set 中弹出 f 值最小的节点进行扩展，若该节点恰好为目标点，则通过父节点指针回溯得到完整路径并返回；否则生成该节点的所有合法邻居，并对每个邻居计算从起点经由当前节点到达的新 g 值，若该值优于之前记录的最佳 g 值，则更新邻居的 g 值、重新计算启发式 h 值、设置父节点为当前节点，并将其加入 open_set；循环持续进行直至 open_set 为空（表示无可行路径）或成功找到目标节点。

八、优缺点分析

首先，由于采用了可采纳的启发式函数，算法在四方向或八方向移动下能够保证找到最短路径，满足路径规划的最优性要求；其次，通过参数化启发式和移动方向，代码具备了良好的灵活性，可根据地图特征和运动约束选择最合适的配置；再者，模块化的设计使得代码结构清晰、易于阅读和扩展，例如新增启发式函数只需在 heuristic 方法中添加分支，无需改动核心逻辑。

然而，实现也存在一些固有局限：在内存方面，需要同时维护开放集和封闭集，对于大规模高分辨率地图可能消耗较多内存；启发式函数的选择直接影响搜索效率，若使用欧几里得距离作为四方向移动的启发式，由于该启发式可能高估实际代价，会破坏可采纳性，导致算法失去最优性保证，但本实现允许用户自行选择，需要使用者根据移动方向合理搭配；此外，当前算法假设所有可通行区域代价均一，未考虑不同地形的通行成本差异，也无法动态响应环境变化，每次规划均基于静态地图独立执行。